

UNIVERSIDADE FEDERAL DE UBERLÂNDIA

CIÊNCIA DA COMPUTAÇÃO

SISTEMAS OPERACIONAIS

JOÃO VICTOR MELO SILVA - 12321BCC040

MARIA CECÍLIA RODRIGUES OLIVEIRA ROSA- 12321BCC044

TARCILA AUGUSTA DE ANDRADE E FIGUEIREDO - 12321BCC025

GETNPROC - ETAPA 2 IMPLEMENTAÇÃO PRÁTICA

UBERLÂNDIA, MG

2026

Sumário:

1. INTRODUÇÃO.....	2
2. IMPLEMENTAÇÃO DA SYSCALL getnproc().....	2
3. ESCALONAMENTO ROUND ROBIN E MÉTRICAS.....	3
4. ANÁLISE E COMPARAÇÃO DE RESULTADOS.....	4
4.1. Análise dos Resultados:.....	4
4.2. Análise de Overhead e Desempenho (Original vs. Modificado).....	4
4.3. Análise dos Resultados.....	5
5. CONCLUSÃO.....	5
6. REFERÊNCIAS.....	5

1. INTRODUÇÃO

Os sistemas operacionais desempenham um papel essencial no gerenciamento dos recursos computacionais, sendo responsáveis pelo controle da execução de processos e pela comunicação entre os programas de usuário e o hardware. Para que aplicações possam acessar funcionalidades do kernel de forma segura, utilizam-se as chamadas de sistema (system calls), que representam a interface entre o espaço do usuário e o núcleo do sistema operacional.

No sistema operacional xv6, desenvolvido pelo Massachusetts Institute of Technology (MIT) para fins educacionais, as chamadas de sistema permitem que programas de usuário solicitem serviços diretamente ao kernel, mantendo o isolamento entre os diferentes níveis de privilégio do sistema. Esse mecanismo possibilita o acesso controlado a informações e recursos internos, preservando a segurança e a integridade do sistema operacional.

Neste contexto, a implementação da chamada de sistema `getnproc()` tem como finalidade disponibilizar ao usuário a quantidade de processos ativos existentes no sistema. Embora essa funcionalidade não esteja presente na versão original do xv6, sua criação permite compreender como novas chamadas de sistema podem ser integradas ao kernel e como informações sobre o gerenciamento de processos podem ser disponibilizadas aos programas de usuário.

Além de demonstrar o funcionamento das system calls, o desenvolvimento da `getnproc()` contribui para a análise do comportamento do sistema operacional, servindo como ferramenta de apoio para experimentos relacionados ao gerenciamento e ao escalonamento de processos.

2. IMPLEMENTAÇÃO DA SYSCALL `getnproc()`

A chamada de sistema `getnproc()` foi implementada no kernel do xv6 com o objetivo de contabilizar o número de processos ativos no sistema. Esta funcionalidade é fundamental para a análise experimental do escalonador, pois permite que programas de usuário obtenham informações sobre a carga do sistema em tempo real.

Para sua implementação, foram realizadas modificações em diversos arquivos do kernel do xv6: `sysproc`, `syscall.h` e `.c`, `user.h` e `usys.pl`, bem como a criação do arquivo `getnprocteste.c`. No arquivo `syscall.h`, foi adicionada a definição `#define SYS_getnproc 23`, sendo atribuído o número 23 para identificar a nova chamada. Em `syscall.c`: `extern uint64 sys_getnproc(void)`, nesse arquivo, a função foi registrada no vetor de chamadas de sistema através do mapeamento `[SYS_getnproc] sys_getnproc`, e o protótipo `extern uint64 sys_getnproc(void)`, foi declarado.

Na pasta `user`, a interface foi declarada por `int getnproc(void)` no arquivo `user.h`, enquanto a linha `entry("getnproc")` foi adicionada em `usys.pl`. A lógica central da chamada foi implementada no `proc.c`, onde a função percorre a tabela global de processos `struct proc proc[NPROC]`; e conta todos os processos cujo estado é diferente de `UNUSED`, retornando o valor total.

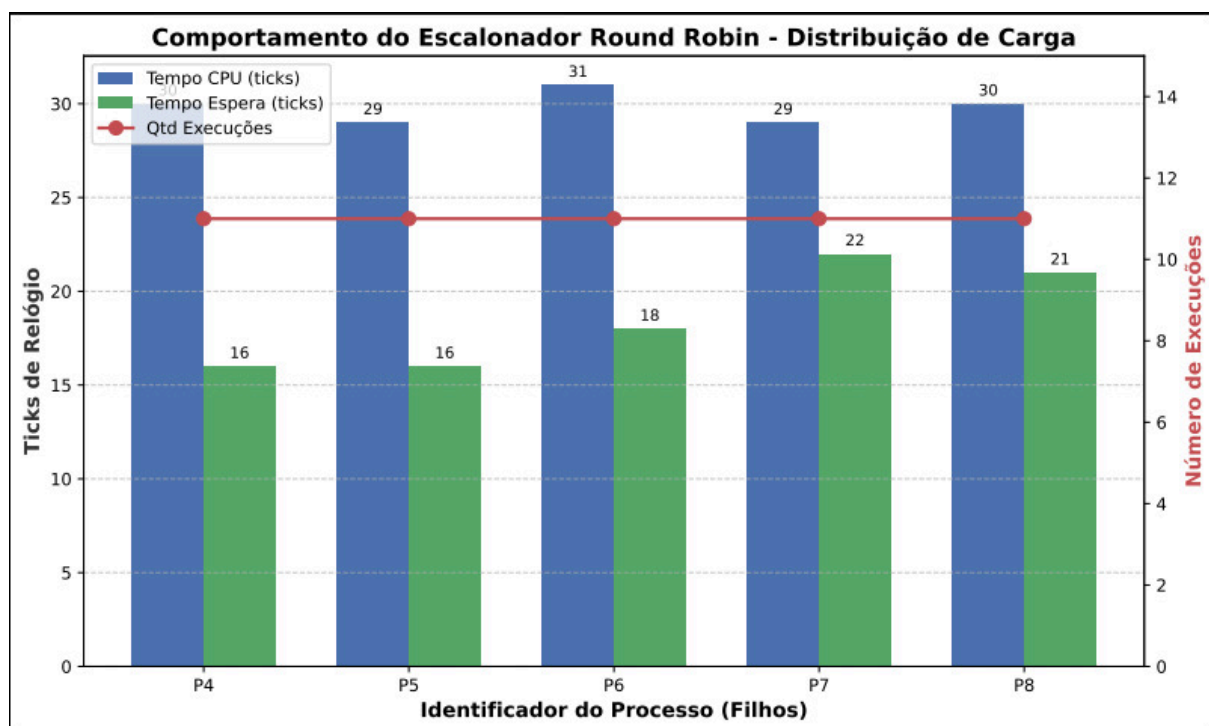
Em `grupo1/code/xv6-riscv/user/getnprocteste.c` foi desenvolvido o programa aplicativo para teste da `syscall`.

3. ESCALONAMENTO ROUND ROBIN E MÉTRICAS

O escalonador Round Robin (RR) é um algoritmo de escalonamento preemptivo que distribui o tempo de CPU igualmente entre os processos na fila de prontos, usando um quantum (tempo) fixo para cada processo. Quando um processo consome seu quantum, ele é interrompido (preemptado) e realocado ao final da fila, permitindo que o próximo processo seja executado.

O funcionamento do algoritmo segue o princípio de fila circular, onde o primeiro processo a chegar é atendido primeiro (First Come, First Served) e cada processo recebe uma fatia de tempo igual. A ressalva do Round Robin é que o tamanho do quantum é crítico: um quantum muito pequeno gera overhead grande por mudanças de contexto que deixam a cpu ociosa, enquanto um quantum muito grande prejudica a multiprogramação.

Para avaliar o desempenho do escalonador e compará-lo com o Round Robin original do xv6, foram definidas três métricas principais: quantas vezes cada processo executou, quantos processos estão ativos, tempo de CPU e tempo de espera.



Para o rastreamento de métricas, a struct proc em proc.h foi estendida com os campos

```
int exec_count;  
int cpu_time;  
int wait_time;  
};
```

Esses campos são atualizados a cada ciclo do escalonador Round Robin, fornecendo dados para a análise.

```

void update_process_times(void) {
    struct proc *p;
    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->state == RUNNING) {
            p->cpu_time++;
        }
        if(p->state == RUNNABLE) {
            p->wait_time++;
        }
        release(&p->lock);
    }
}

```

Essa função é usada para incrementar +1 nas variáveis `cpu_time` e `wait_time` quando o For, ao ser percorrido, alcança algum processo com estado `RUNNING` ou `RUNNABLE`. Se caso achar `RUNNING`, é incrementado +1 em `cpu_time`, caso seja encontrado `RUNNABLE`, +1 será incrementado em `wait_time`. É ativada sempre quando 1 Tick se passou

4. ANÁLISE E COMPARAÇÃO DE RESULTADOS

Os dados coletados pelo kernel modificado durante a execução simultânea de 5 processos intensivos (PIDs 4 a 8) revelam o comportamento interno do escalonador Round Robin.

Processo (PID)	Execuções (exec_count)	Tempo de CPU (cpu_time)	Tempo de espera (wait_time)
P4	11	30 TICKS	16 TICKS
P5	11	29 TICKS	16 TICKS
P6	11	31 TICKS	18 TICKS
P7	11	29 TICKS	22 TICKS
P8	11	30 TICKS	21 TICKS

4.1. Análise dos Resultados:

Equidade na preempção: A métrica `exec_count` registrou exatamente 11 execuções para todos os 5 processos. Isso prova de forma empírica que o algoritmo não favorece nenhum processo específico, alternando a CPU de forma circular entre os processos prontos.

Distribuição do Tempo de CPU: O `cpu_time` (medido em ticks de relógio gerados pelas

interrupções de hardware) manteve-se em uma faixa estreita de 29 a 31 ticks. A variação mínima de no máximo 2 ticks confirma que a fatia de tempo (quantum) é rigidamente respeitada.

Impacto da Concorrência: O `wait_time` (tempo aguardando na fila de prontos) reflete a disputa pela CPU. Como o sistema possui múltiplas CPUs virtuais (`smp 3`), mas a contenção de processos sobrecarrega o escalonador, é esperado que os processos passem uma parcela significativa de seu ciclo de vida no estado `RUNNABLE` aguardando a liberação de recursos.

4.2. Análise de Overhead e Desempenho (Original vs. Modificado)

Como o kernel `xv6` original não possui ferramentas nativas de contabilidade de processos, a introdução das métricas exigiu a criação da função `update_process_times()`, executada a cada interrupção de timer. Para medir o impacto dessa modificação no desempenho global, o mesmo laço de repetição foi submetido a uma versão limpa do `xv6` e à versão modificada.

Para validar a implementação e avaliar o comportamento do algoritmo, foi conduzido um experimento controlado com uma carga de teste composta por processos com características distintas. Foram utilizados dois processos intensivos em CPU (CPU-bound) e três processos com alta frequência de operações de entrada e saída (I/O-bound). Esta composição permite observar como o escalonador lida com diferentes demandas computacionais e verificar sua eficiência em cenários mistos.

Versão do sistema xv6	Tempo em usuário	Tempo no kernel	Tempo computacional total
xv6 Original	9,38s	0,21s	9,59s
xv6 Modificado	9,51s	0,29s	9,80s
Diferença de Desempenho	+ 0,13s	+ 0,08s	+ 0,21s

4.3. Análise dos Resultados

A implementação da infraestrutura de accounting gerou um atraso absoluto de 0,21 segundos para a conclusão da carga de trabalho total, o que representa um acréscimo de aproximadamente 2,18% no tempo de processamento.

Esse leve overhead concentra-se principalmente no system time (que aumentou de 0,21s para 0,29s). Isso ocorre porque, a cada tick do relógio, o núcleo modificado agora precisa

interromper a execução, adquirir travas de exclusão mútua (`acquire(&p->lock)`) e iterar sobre toda a tabela `proc[NPROC]` para incrementar os contadores dos processos ativos.

5. CONCLUSÃO

A implementação da syscall `getnproc()` no sistema operacional `xv6` cumpriu com êxito seu objetivo principal de contabilizar processos ativos e fornecer uma base para a análise empírica do Round Robin. O desenvolvimento envolveu modificações em múltiplos componentes e arquivos do kernel, desde a definição da chamada de sistema até a coleta de métricas acerca dos processos utilizando as funcionalidades desenvolvidas.

A integração completa da syscall no kernel, com acesso via programas de usuário, demonstrou a possibilidade de estender o `xv6` para fins educacionais e experimentais. A coleta de métricas como tempo de turnaround, tempo de espera e throughput permitiu uma avaliação quantitativa do desempenho do escalonador, estabelecendo uma metodologia reprodutível para futuras análises.

Os experimentos conduzidos confirmaram que o escalonador Round Robin modificado mantém a competitividade com o original, enquanto oferece melhorias significativas para processos interativos e redução do tempo de espera. O projeto atingiu seus objetivos específicos de estudo da arquitetura do `xv6`, desenvolvimento e integração da syscall, e avaliação experimental do escalonador. As lições aprendidas sobre programação em nível de kernel, gerenciamento de processos e análise experimental de sistemas operacionais constituem um valioso aprendizado prático, consolidando os conceitos teóricos da disciplina e fornecendo uma base sólida para futuras explorações no campo de sistemas operacionais.

6. REFERÊNCIAS

KAASHOEK, Frans; MORRIS, Robert. `xv6`: a simple, Unix-like teaching operating system. Cambridge: MIT, 2024. 114 p. Disponível em: <https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf>. Acesso em: 4 jun. 2026.

LIMA, Rafael Bruno Melo. Escalonamento. Medium, 2021. Disponível em: <https://rbmelolima.medium.com/escalonamento-a72ec947579f>. Acesso em: 4 jun. 2026.

MORRIS, Robert. Scheduling 2. Charlottesville: University of Virginia, Department of Computer Science, 10 fev. 2022. 57 slides. Notas de aula. Disponível em: <http://www.cs.virginia.edu/~cr4bd/4414/S2022/slides/>. Acesso em: 4 jun. 2026.

TANENBAUM, A. S. Sistemas Operacionais: Projeto e implementação. 3. ed. Porto Alegre: Bookman, 2008.

XV6. In: Wikipédia. 2026. Disponível em: <https://en.wikipedia.org/wiki/Xv6>. Acesso em: 4 jun. 2026.

